Create DataFrame

```
data = [
    (1, "Rahul Sharma", "Hyderabad", "Electronics", "Laptop", 1, 65000, '
    (2, "Priya Reddy", "Bangalore", "Electronics", "Mobile", 2, 25000, "I
    (3, "Amit Kumar", "Mumbai", "Furniture", "Chair", 4, 3500, "Pending")
    (4, "Sneha Patel", "Chennai", "Electronics", "Tablet", 1, 30000, "Car
    (5, "Farhan Ali", "Delhi", "Furniture", "Table", 2, 12000, "Delivered
    (6, "Neha Singh", "Hyderabad", "Fashion", "Shoes", 3, 2500, "Delivere
    (7, "Arjun Verma", "Pune", "Fashion", "Watch", 1, 8000, "Pending"),
    (8, "Meera Nair", "Bangalore", "Electronics", "TV", 1, 45000, "Delive
    (9, "Kiran Rao", "Hyderabad", "Furniture", "Desk", 1, 15000, "Deliver
    (10, "Ravi Kumar", "Mumbai", "Electronics", "Mobile", 1, 28000, "Deli
]

columns = [
    "order_id",
    "customer_name",
    "city",
    "category",
    "product",
    "quantity",
    "price",
    "order_status"
]

df = spark.createDataFrame(data, columns)

df.show()
```

Exercises

1. Display all orders.

2. Display only customer_name, city and product.

3. Display only product and price.

4. Display all orders from Hyderabad.

5. Display all orders from Bangalore.

6. Display all Electronics orders.

7. Display all Furniture orders.

8. Display orders where price is greater than 30000.

9. Display orders where quantity is greater than 1.

10. Display delivered orders.

11. Display pending orders.

12. Display cancelled orders.

13. Display orders from Hyderabad or Mumbai.

14. Display orders where category is Electronics or Fashion.

15. Display orders where price is between 10000 and 50000.

16. Sort orders by price ascending.

17. Sort orders by price descending.

18. Sort orders by city and price.

19. Create a new column total_amount = quantity * price.

20. Create a new column discount = total_amount * 0.10.

21. Create a new column final_amount = total_amount - discount.

22. Rename customer_name to customer.

23. Rename order_status to status.

24. Display total revenue using total_amount.

25. Display average order value.

26. Display highest order value.

27. Display lowest order value.

28. Count total orders.

29. Count orders by city.

30. Count orders by category.

31. Count orders by order_status.

32. Find total revenue by city.

33. Find total revenue by category.

34. Find total revenue by product.

35. Find average price by category.

36. Find maximum price by category.

37. Find minimum price by category.

38. Display unique cities.

39. Display unique categories.

40. Display unique order statuses.

41. Display top 3 highest value orders.

42. Display lowest value order.

43. Display products whose name starts with 'M'.

44. Display customers whose name contains 'a'.

45. Display orders where product contains 'o'.

46. Create order_value_category:
    total_amount >= 50000 -> High
    total_amount >= 20000 -> Medium
    Else -> Low

47. Count orders by order_value_category.

48. Display only delivered orders with final_amount.

49. Display city-wise delivered order revenue.

50. Generate final report with:
    order_id
    customer_name
    city
    category
    product
    quantity
    price
    total_amount
    discount
    final_amount
    order_status